

제13강

1) Firebase Firestore Database을 이용한 장바구니 등록, 삭제

- [Firebase 내맵]-[제품 카테고리]-[Firestore Database]-[규칙]에서 rules의 read, write을 true로 변경한다.

[src]-[components13]-[shop] ShopPage.jsx

```
import Cart from './shop/Cart'
...
const ShopPage = () => {
  return (
    <Card.Body className='position-relative'>
      <ShopModal shop={shop}/>
      <Cart shop={shop}/>
    </Card.Body>
  )
}
```

[src]-[components13]-[shop] Cart.jsx

```
import { app } from '../initFirebase'
import { getFirestore, collection, query, where, getCountFromServer, addDoc } from 'firebase/firestore'
...
const Cart = ({shop}) => {
  const navi = useNavigate();
  const { setAlert } = useContext(ModalContext);
  const db = getFirestore(app);
  const uid = sessionStorage.getItem('uid');

  const onRegister = async() => {
    if(uid){
      const q = query(collection(db, 'cart'), where('uid', '==', uid), where('productId','==',shop.productId));
      const snapshot = await getCountFromServer(q);
      if(snapshot.data().count > 0){
        setAlert({show:true, message:'이미 장바구니에 있는 상품입니다.'});
      }else{
        await addDoc(collection(db, 'cart'), { uid, ...shop });
        setAlert({show:true, message:'장바구니에 등록되었습니다.'});
      }
    }else{
      navi('/login');
    }
  }

  return (
    <div className='position-absolute bottom-0 end-0 me-1 icon-cart'>
      <FaCartPlus onClick={ onRegister }/>
    </div>
  )
}
```

2) Firebase Firestore Database을 이용한 장바구니 목록

[src]-[components13] MainRouter.jsx

```
import CartPage from './shop/CartPage'
...
const MainRouter = () => {
  <Routes>
    ...
    <Route path='favorite' element={}<FavoritePage/>>/>
    <Route path='/'<b>cart</b>' element={}<CartPage/>>/>
  </Routes>
}
```

[src]-[components13] CartPage.jsx

```
import { useContext, useEffect, useState } from 'react';
import { app } from '../initFirebase'
import { getFirestore, collection, query, where, onSnapshot, deleteDoc, doc } from 'firebase/firestore'
import { Table, Button } from 'react-bootstrap'
import { ModalContext } from '../context/ModalContext'

const CartPage = () => {
  const { setConfirm } = useContext(ModalContext);
  const [loading, setLoading] = useState(false);
  const [products, setProducts] = useState([]);
  const db = getFirestore(app);
  const uid = sessionStorage.getItem('uid');

  const callAPI = () => {
    setLoading(true);
    const q = query(collection(db, 'cart'), where('uid','==',uid));
    onSnapshot(q, snapshot=>{
      const rows = [];
      snapshot.docs.forEach(row=>{
        rows.push({ id:row.id, ...row.data() });
      });
      console.log(rows);
      setProducts(rows);
    });
    setLoading(false);
  }

  useEffect(()=>{
    callAPI();
  },[]);

  const onDelete = (product) => {
    setConfirm({
      show:true,
      message:`${product.id} ' 상품을 삭제하실래요?',
      action: async()=>{
        await deleteDoc(doc(db, 'cart', product.id));
      }
    });
  }

  if(loading) return <h1 className='text-center my-5'>로딩중...</h1>
  return (
    <div className='my-5'>
      <h1 className='text-center mb-5'>장바구니</h1>
      <Table hover striped>
        <tbody>
          {products.map((product, index)=>
            <tr key={index}>
              <td>{index+1}</td>
              <td><div dangerouslySetInnerHTML={{__html:product.title}}</div>
              <td className='text-end'>{ parseInt(product.lprice).toLocaleString() }원</td>
              <td>{product.maker}/{product.mallName}</td>
              <td><Button onClick={ ()=>onDelete(product) } variant='outline-danger' size='sm'>삭제</Button></td>
            </tr>
          )}
        </tbody>
      </Table>
    </div>
  )
}

export default CartPage
```